

CompassLabs

Backend — Python Package Reference

Every installed package · why it's here · what layer it belongs to · what we chose NOT to install and why

1. Install Command (Run Once in venv)

```
(venv) PS D:\AI\CompassLabs\backend> pip install fastapi uvicorn sqlalchemy alembic \
    pydantic[email] python-jose[cryptography] passlib[bcrypt] \
    httpx python-dotenv redis groq
```

That single command installs all 11 packages and their dependencies into your virtual environment. Nothing else should be installed at MVP stage.

2. Full Package Reference — What Each One Does

Package	Layer	Install Name	What It Does in CompassLabs
fastapi	Server	fastapi	Our web framework — defines all API routes (/auth, /company, /compare)
uvicorn	Server	uvicorn	The ASGI server that actually runs FastAPI in dev and production
sqlalchemy	Database	sqlalchemy	ORM — lets us write Python classes instead of raw SQL for PostgreSQL
alembic	Database	alembic	Handles DB migrations — tracks every change to our table schemas
pydantic[email]	Validation	pydantic[email]	Validates all incoming request data + email format checking
python-jose[cryptography]	Auth	python-jose[cryptography]	Creates and verifies JWT access/refresh tokens
passlib[bcrypt]	Auth	passlib[bcrypt]	Hashes passwords with bcrypt — never stores plain text passwords
httpx	HTTP	httpx	Async HTTP client — calls Wikipedia, NewsAPI, Yahoo Finance etc.
python-dotenv	Config	python-dotenv	Reads .env file — loads API keys and secrets into the app
redis	Cache	redis	Connects to Upstash Redis — reads/writes our 6-hour company cache
groq	LLM	groq	Official Groq SDK — sends prompts to LLaMA 3 70B, gets JSON insights back

3. Layer Map — How Packages Group Together

Each package belongs to a specific layer of the backend architecture. Understanding this prevents you from calling the wrong package in the wrong place.

Layer	Packages	Responsibility
Server	<code>fastapi</code> , <code>uvicorn</code>	Receives HTTP requests, routes them, sends responses back
Database	<code>sqlalchemy</code> , <code>alembic</code>	Talks to PostgreSQL — stores users, reviews, bookmarks permanently
Validation	<code>pydantic[email]</code>	Checks every request/response shape — wrong data never reaches business logic
Auth	<code>python-jose[cryptography]</code> , <code>passlib[bcrypt]</code>	Issues JWT tokens, verifies them, hashes/checks passwords
HTTP	<code>httpx</code>	Fetches data from Wikipedia, NewsAPI, Yahoo Finance, Wikidata in parallel
Config	<code>python-dotenv</code>	Loads <code>.env</code> secrets into app — keeps API keys out of code
Cache	<code>redis</code>	Reads/writes Upstash Redis — 6-hour TTL protects our free API quotas
LLM	<code>groq</code>	Sends merged company data to LLaMA 3 70B, gets structured JSON insights back

4. The Rule — Which Layer Calls Which

Router → Service → Data Source / DB / Cache / LLM

- Routers (`fastapi`) receive the request and call a service. They never touch the DB or LLM directly.
- Services contain business logic. They call `sqlalchemy` (DB), `httpx` (external APIs), `redis` (cache), or `groq` (LLM).
- `Pydantic` validates data at the boundary — before it enters a router, and before it leaves in a response.
- `python-dotenv` loads secrets on startup — all other packages read from settings, never from `os.environ` directly.

5. What We Are NOT Installing (and Why)

Intentional decisions — keeping the backend lean at MVP. Everything below is either a future phase item or simply not needed for this stack.

Package	Category	Why NOT Installed
Django	Full-stack web framework	Too heavy, brings ORM+admin+auth we don't need — FastAPI is leaner
Flask	Micro web framework	No async support by default — httpx + asyncio needs async-first framework
requests	HTTP client (sync)	Replaced by httpx — requests is synchronous, httpx supports async

aiohttp	Async HTTP client	htpx is simpler API with same async power — no reason for both
celery	Task queue	Not needed at MVP — no background jobs yet (V2+ feature)
langchain	LLM orchestration	Overkill for MVP — direct Groq SDK is simpler and faster to learn
stripe	Payments	V3 feature — no billing at MVP stage
boto3	AWS SDK	We use Supabase + Upstash, not AWS services
pandas / numpy	Data analysis	No ML/analytics at MVP — V3 salary analytics feature
pytest	Testing framework	Will add in V2 once core features stabilize

6. requirements.txt — Dependency Lock File

Generated by running: `pip freeze > requirements.txt` after installing packages. This pins exact versions so every developer (and Render.com deployment) installs identical packages.

```
# To recreate the environment from scratch:
(venv) PS D:\AI\CompassLabs\backend> pip install -r requirements.txt
```

Never manually edit requirements.txt. Always regenerate it with pip freeze after adding or removing a package.